

DBMS – DATABASE SECURITY AND PERFORMANCE OPTIMIZATION

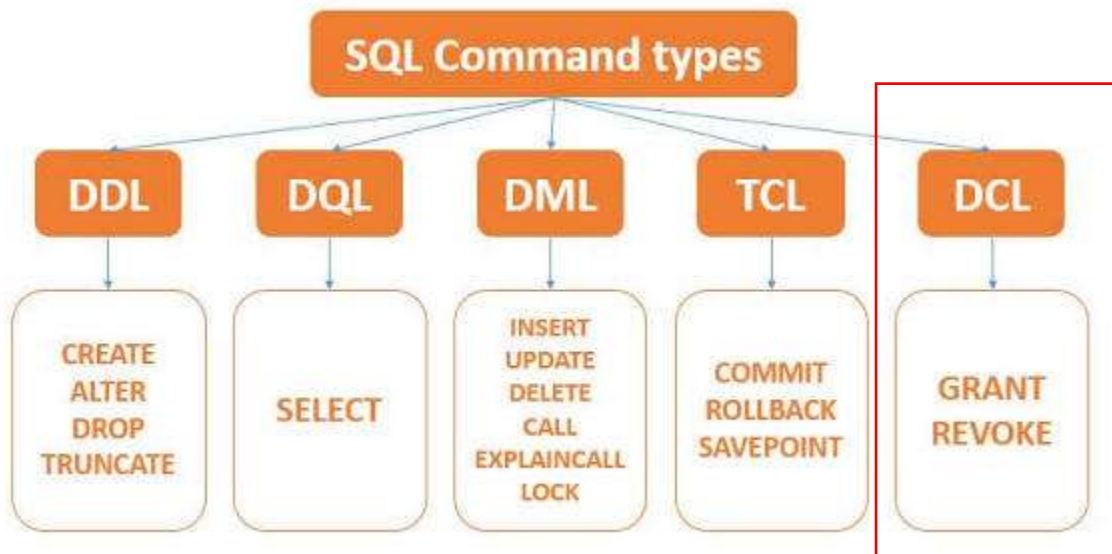
Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

Skill Task: DBMS

Week: 13

Objectives

1. Create database users and roles.
2. Implement GRANT and REVOKE commands.
3. Apply access control.
4. Create indexes for faster searching.
5. Analyze and optimize query performance.



1. Create Database Users and Roles

To create database users and assign appropriate roles for accessing the EcoCart database.

Step 1: Create a User

We need to execute user-creation commands using an account with sufficient privileges such as SYSTEM.

```
SQL> CREATE USER ecocart_user  
2 IDENTIFIED BY EcoCart123;  
  
User created.
```

Step 2: Allow Login

```
SQL> GRANT CREATE SESSION  
2 TO ecocart_user;  
  
Grant succeeded.
```

CREATE SESSION allows the user to connect to the Oracle database.

Step 3: Create a Role

Instead of individually granting permissions to every user, create a role.

```
SQL> CREATE ROLE ecocart_role;  
  
Role created.
```

Step 4: Grant Permissions to the Role

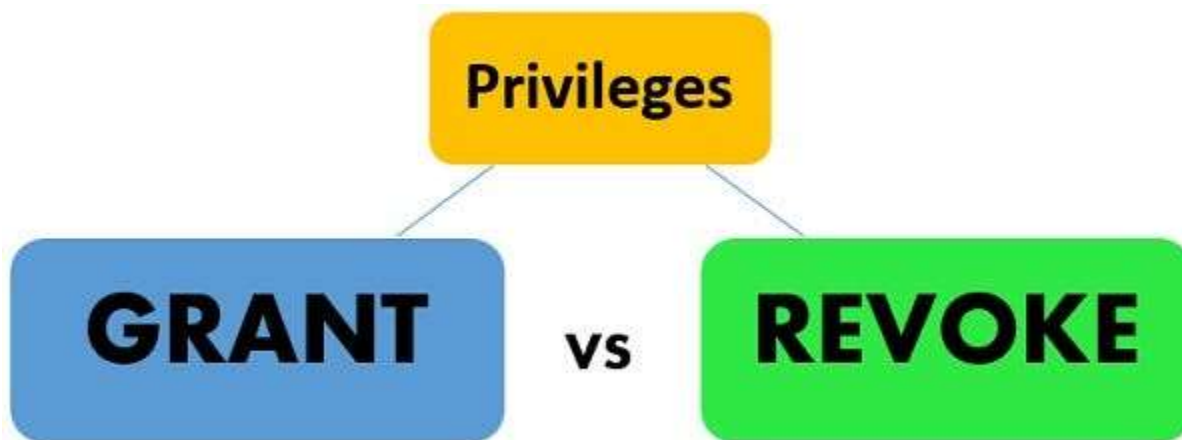
For example, allow the role to view customer and order information:

```
SQL> GRANT SELECT ON customer  
2 TO ecocart_role;  
  
Grant succeeded.  
  
SQL> GRANT SELECT ON orders  
2 TO ecocart_role;  
  
Grant succeeded.  
  
SQL> GRANT SELECT ON payment  
2 TO ecocart_role;  
  
Grant succeeded.
```

Step 5: Assign Role to User

```
SQL> GRANT ecocart_role  
2 TO ecocart_user;  
  
Grant succeeded.
```

2. Implement GRANT and REVOKE Commands



GRANT

The GRANT command in sql is defined to giving the access or privileges to the users.

Three components are used in the authorization are users,privilege (operation) & a database object.

```
SQL> GRANT SELECT, INSERT  
2 ON payment  
3 TO ecocart_role;  
  
Grant succeeded.
```

Now users having ecocart_role can:

- View payment records.
- Insert payment records.

```
SQL> SELECT *
2 FROM payment;
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS
501	201	UPI	16-SEP-26	2500	Successful
502	202	Card	16-SEP-26	1800	Successful
503	203	Net Banking	16-SEP-26	3200	Successful
504	204	UPI	16-SEP-26	1500	Successful
1	109	UPI	06-OCT-26	5000	SUCCESS
505	201	UPI	06-OCT-26	2500	Successful

6 rows selected.

REVOKE

The REVOKE command in sql is defined to take away the granted privileges from the user.

The authority to withdraw the privileges is the database administrator.

```
SQL> REVOKE INSERT
2 ON payment
3 FROM ecocart_role;
```

Revoke succeeded.

Now the role can still perform:

```
SQL> SELECT *
2 FROM payment;
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT
501	201	UPI	16-SEP-26	2500
502	202	Card	16-SEP-26	1800
503	203	Net Banking	16-SEP-26	3200
504	204	UPI	16-SEP-26	1500
1	109	UPI	06-OCT-26	5000
505	201	UPI	06-OCT-26	2500

6 rows selected.

but cannot insert new payment records.

3. Apply Access Control

To restrict users from accessing tables or performing operations that are not required for their role.

For example, suppose an **Order Manager** should view orders and payments but should not modify customer information.

Create:

```
SQL> CREATE ROLE order_manager;

Role created.

SQL> GRANT SELECT, INSERT
2 ON orders
3 TO order_manager;

Grant succeeded.
```

Grant only the required privileges:

```
SQL> GRANT SELECT
2  ON payment
3  TO order_manager;

Grant succeeded.
```

Create a user:

```
SQL> CREATE USER order_staff
2  IDENTIFIED BY Order123;

User created.
```

Allow login:

```
SQL> GRANT CREATE SESSION
2  TO order_staff;

Grant succeeded.
```

Assign the role:

```
SQL> GRANT order_manager
2  TO order_staff;

Grant succeeded.
```

The user gets only the permissions provided through order_manager.

This demonstrates **role-based access control**.

4. Create Indexes for Faster Searching

To create indexes on frequently searched columns to improve query performance.

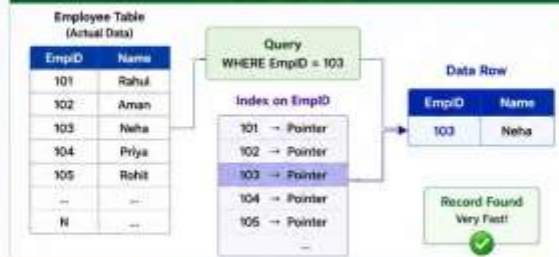
An index is a database object that helps Oracle locate rows more efficiently.

SQL INDEX EXPLAINED

Why Use Index?



How Index Works



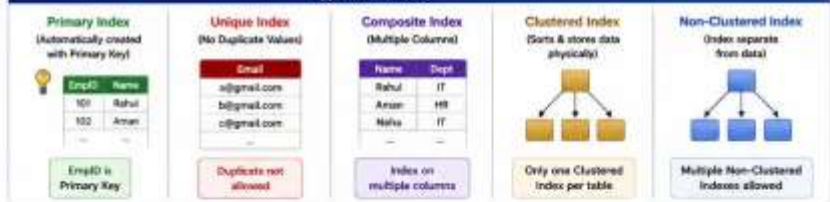
Full Table Scan vs Index Scan



CREATE INDEX Examples



Types of Indexes



Index Structure (Example)



Clustered vs Non-Clustered Index



Advantages / Disadvantages



When to Use Index?



Example

Suppose orders are frequently searched using CUSTOMER_ID.

```
SQL> SELECT *
2 FROM orders
3 WHERE customer_id = 101;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT
201	101	16-SEP-26	2700
101	101	06-OCT-26	5000
105	101	06-OCT-26	5000
109	101	06-OCT-26	5000

Create an index:

```
SQL> CREATE INDEX idx_orders_customer
2 ON orders(customer_id);
```

Index created.

Now Oracle has an index on CUSTOMER_ID.

Payment Index

Payment records may frequently be searched using ORDER_ID.

```
SQL> CREATE INDEX idx_payment_order
  2  ON payment(order_id);

Index created.
```

Inventory Index

Inventory may frequently be searched using PRODUCT_ID.

```
SQL> CREATE INDEX idx_inventory_product
  2  ON inventory(product_id);

Index created.
```

Check Indexes

```
SQL> SELECT index_name,
  2      table_name,
  3      column_name
  4  FROM user_ind_columns
  5  WHERE table_name IN
  6      ('ORDERS', 'PAYMENT', 'INVENTORY');
```

INDEX_NAME	TABLE_NAME
COLUMN_NAME	
SYS_C007046 INVENTORY_ID	INVENTORY
IDX_INVENTORY_PRODUCT PRODUCT_ID	INVENTORY
SYS_C007079 ORDER_ID	ORDERS

INDEX_NAME	TABLE_NAME
COLUMN_NAME	
IDX_ORDERS_CUSTOMER CUSTOMER_ID	ORDERS
SYS_C007091 PAYMENT_ID	PAYMENT
IDX_PAYMENT_ORDER ORDER_ID	PAYMENT

5. Analyze and Optimize Query Performance

We can compare the execution plan **before and after creating an index**.

Step 1: Display Execution Plan

```
SQL> EXPLAIN PLAN FOR
2  SELECT order_id,
3         order_date,
4         total_amount
5  FROM orders
6  WHERE customer_id = 101;
```

Explained.

Then:

```
SQL> SELECT *
2  FROM TABLE(DBMS_XPLAN.DISPLAY);
```

PLAN_TABLE_OUTPUT

Plan hash value: 1275100350

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	19	2 (0)	00:00:01
* 1	TABLE ACCESS FULL	ORDERS	1	19	2 (0)	00:00:01

Predicate Information (identified by operation id):

PLAN_TABLE_OUTPUT

1 - filter("CUSTOMER_ID">=101)

13 rows selected.

This shows Oracle's planned method for retrieving the data.

Step 2: Create Index

```
SQL> CREATE INDEX idx_orders_customer
2  ON orders(customer_id);
```

Index created.

Step 3: Generate the Plan Again

```
SQL> EXPLAIN PLAN FOR
2  SELECT *
3  FROM orders
4  WHERE customer_id = 101;

Explained.

SQL> SELECT *
2  FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
-----
Plan hash value: 1275100350

-----
| Id | Operation          | Name | Rows | Bytes | Cost (%CPU)| Time     |
-----+-----+-----+-----+-----+-----+-----+
|  0 | SELECT STATEMENT   |      |     1 |    19 |        2 (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| ORDERS|     1 |    19 |        2 (0)| 00:00:01 |
-----+-----+-----+-----+-----+-----+

Predicate Information (identified by operation id):
-----

PLAN_TABLE_OUTPUT
-----

      1 - filter("CUSTOMER_ID">=101)

13 rows selected.
```

Now we can compare the execution plans.

Before Index

Oracle may need to perform:

TABLE ACCESS FULL

This means Oracle examines many/all rows in the table.

After Index

Oracle may use:

INDEX RANGE SCAN

This means Oracle can use the index to locate matching rows more efficiently.

6. Query Optimization Examples

Example 1 – Avoid unnecessary columns

Instead of:

```
SQL> SELECT *
2 FROM orders
3 WHERE customer_id = 101;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT
201	101	16-SEP-26	2700
101	101	06-OCT-26	5000
105	101	06-OCT-26	5000
109	101	06-OCT-26	5000

```
SQL> SELECT order_id,
2         order_date,
3         total_amount
4 FROM orders
5 WHERE customer_id = 101;
```

ORDER_ID	ORDER_DAT	TOTAL_AMOUNT
201	16-SEP-26	2700
101	06-OCT-26	5000
105	06-OCT-26	5000
109	06-OCT-26	5000

Only required columns are retrieved.

Example 3 – Payment Search

```
SQL> CREATE INDEX idx_payment_order
2 ON payment(order_id);
```

Index created.

```
SQL> SELECT payment_id,
2         payment_mode,
3         payment_amount,
4         payment_status
5 FROM payment
6 WHERE order_id = 201;
```

PAYMENT_ID	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
501	UPI	2500	Successful
505	UPI	2500	Successful